Project Proposal: Extracting Implied Constraints from BTOR2 Models

MOSTAFA REZVANIFAR s5310938
Mingjia Li

September 24, 2026

1 Introduction

Model checking is a powerful automated reasoning technique for verifying that hardware designs behave correctly before they are manufactured. Modern model checkers rely on SAT and SMT solvers, which are among the most successful tools of symbolic artificial intelligence. However, their scalability remains limited, since the state space to be explored grows exponentially with the size of the design. A classical remedy is to make hidden knowledge explicit. Constraints that are implied by the model or by the property, but never stated, narrow the verification problem to a subset of the system states without changing whether the property holds [1, 2]. In this way, they prune the search, much as constraint propagation and learned clauses do in constraint programming and SAT solving.

The remainder of this proposal is organized as follows. Section 2 states the problem, Section 3 discusses related work, Section 4 outlines the project planning.

2 Problem Statement

The goal of this project is to extract word-level constraints that are logically implied by the safety properties or by the structure of a BTOR2 transition system. Such constraints are not required to be explicitly present in the original model, but they must be consequences of information that is already available in the model or in the property.

Given a BTOR2 model and a safety property P, the extractor generates candidate constraints C over the model's bit-vector and array expressions. The main candidate families considered in this project are bit-vector bounds, equalities, disequalities, implications, masked-bit relations, and array relations. The extraction process should preserve the word-level structure of the BTOR2 model rather than bit-blasting it into a Boolean representation.

A central requirement is that candidate generation itself is not trusted. Every proposed property-derived constraint must therefore be independently validated by an SMT solver. A candidate C is accepted only if $P \Rightarrow C$, which is checked by testing the satisfiability of $P \wedge \neg C$. If this formula is unsatisfiable, then C is proved to be implied by the property. If it is satisfiable, the candidate is rejected; an inconclusive solver result is reported as unknown. Candidates derived from the model structure are validated using an analogous implication obligation in which the relevant model constraints imply the candidate.

The project therefore investigates two main questions: which useful word-level constraints can be generated automatically from BTOR2 properties and model structure, and how often the generated candidates can be formally validated. As a secondary question, we investigate whether validated constraints can be reused across multiple properties and whether they can reduce the cost of subsequent model-checking queries.

3 Related Work

Cabodi et al. [1] showed that exploiting explicit and hidden verification constraints speeds up bit-level model checking. Froleyks et al. [2] classify extracted constraints into model, inductive, and property constraints, and show how each class can be used soundly and certified in bit-level (AIGER) model checking. In particular, they

show that property constraints must be integrated into the transition function rather than added directly. Our approach of first proposing candidates and then proving them is similar to Houdini [5], which guesses candidate invariants and keeps only those that can be proven. In contrast to these works, we target word-level BTOR2 models [4], where constraints can refer directly to bit-vector and array expressions. We build on Pono [3] or another model checker.

4 Project Planning

Several implementation details have not been clearly defined.

4.1 Workflow

M1. BTOR2 parsing. Use the Pono BTOR2 frontend, or another suitable BTOR2 frontend, to parse the selected benchmarks and obtain the states, inputs, initial conditions, transition relations, model constraints, and bad-state properties while preserving bit-vector and array expressions. If the existing frontends do not provide the information required for constraint extraction, a small self-contained BTOR2 reader will be implemented as a fallback.

M2. Property and model analysis. For each bad-state node b, construct the safety property $P = \neg b$ and analyze the relevant property and model expressions that will be used for candidate generation.

M3. Candidate constraint generation. Traverse the analyzed word-level expressions and apply a collection of syntactic generation rules to propose candidate constraints. Property-derived candidates are generated from the structure of the safety property, while model-derived candidates are generated from structural relations in the BTOR2 model. The main candidate families include bit-vector bounds, equalities, disequalities, implications, masked-bit relations, and array relations. Each candidate is recorded together with its family and source so that the different generation methods can be evaluated separately.

M4. SMT validation. Validate each candidate constraint using Z3 through its Python API. For a candidate C derived from a property P, check the satisfiability of $P \wedge \neg C$. An unsatisfiable result validates C as an implied constraint, while satisfiable and inconclusive results are recorded as rejected and unknown, respectively. Bitwuzla may be used as an alternative solver if needed. Candidates derived directly from the model are checked using the corresponding model implication obligation.

M5. Evaluation. Run the workflow on a collection of BTOR2 benchmarks and record, for each model and property, the number and types of generated candidates, validated constraints, rejected and unknown candidates, and the extraction and validation times. If time permits, we also investigate constraint reuse across multiple properties and their effect on model-checking performance.

4.2 Tooling

The implementation language is Python. The primary SMT solver is Z3, accessed through its Python API. Bitwuzla may be used as an alternative solver if needed. For the frontend we will first evaluate the BTOR2 frontend of Pono [3], which is based on the `btor2tools` library and is accessible from Python through Pono's bindings. Since BTOR2 is a flat, one-node-per-line format [4], a self-contained reader is a viable fallback should the toolchain prove difficult to build; the decision will be made during M1 and reported.

4.3 Benchmarks

We use word-level BTOR2 benchmarks from the Hardware Model Checking Competition. Development is carried out on the smallest benchmarks, ordered by file size, so that intermediate results can still be inspected by hand; the full selection is used only once the pipeline runs end to end. In addition, we maintain a small set of hand-written BTOR2 models with known expected results as regression tests for the frontend and the validator.

The selection will cover three groups: models using only bit-vector sorts, models using arrays, and models carrying several bad-state properties. The third group is required for the analysis of constraint reuse across properties; we note that competition editions that split multi-property benchmarks into single-property files are unsuitable for that part of the evaluation, so multi-property models will be sourced accordingly.

4.4 Evaluation Methodology and Metrics

For each model and property we record the theory features used, the number of candidates proposed, validated, rejected, and unknown, broken down by family, and the extraction and validation times. The main metric is the acceptance rate per family, which indicates which classes of word-level candidates are worth generating; a negative result for a family is itself a reportable outcome.

Soundness evidence is collected by construction: every accepted constraint is accompanied by its discharged obligation. We report representative examples for both bit-vector and array models.

If the integration is carried out, we compare verification time and the number of solved properties with and without constraints, and, for multi-property models, count the constraints reused across properties.

References

[1] G. Cabodi, P. Camurati, L. Garcia, M. Murciano, S. Nocco, and S. Quer. Speeding up model checking by exploiting explicit and hidden verification constraints. In *Design, Automation & Test in Europe (DATE)*, pages 1686–1691. IEEE, 2009.

[2] N. Froleyks, E. Yu, A. Biere, and K. Heljanko. Certifying constraints in hardware model checking. In *International Symposium on Formal Methods (FM)*. Springer, 2026.

[3] M. Mann, A. Irfan, F. Lonsing, Y. Yang, H. Zhang, K. Brown, A. Gupta, and C. Barrett. Pono: A flexible and extensible SMT-based model checker. In *Computer Aided Verification (CAV)*, LNCS 12760, pages 461–474. Springer, 2021.

[4] A. Niemetz, M. Preiner, C. Wolf, and A. Biere. BTOR2, BtorMC and Boolector 3.0. In *Computer Aided Verification (CAV)*, LNCS 10981, pages 587–595. Springer, 2018.

[5] C. Flanagan and K. R. M. Leino. Houdini, an annotation assistant for ESC/Java. In *Formal Methods Europe (FME)*, LNCS 2021, pages 500–517. Springer, 2001.